Student Project Report

Project Title: Firewall Implementation (PyWall: A Stateful Packet-Filtering Firewall)

1. Abstract

The proposed project develops PyWall, a stateful packet-filtering firewall for Linux written in Python, to filter packets and enhance network security. The firewall receives every IPv4 and IPv6 packet entering or leaving the host from the Linux kernel through netfilter's NFQUEUE interface, parses the IPv4, IPv6, TCP, UDP, ICMP and ICMPv6 headers directly from raw bytes, and returns a verdict for each packet: ACCEPT, DROP or REJECT.

Each packet passes through a fixed decision pipeline: header parsing, whitelist and blacklist checks, intrusion-detection bans, intrusion detection, connection tracking, an ordered first-match rule table and a default-deny policy. The system detects SYN floods, port scans and ping floods and bans the attacking source, drops spoofed and unsolicited packets, and rate-limits selected traffic. A real-time web dashboard draws the traffic flowing through the pipeline, explains every verdict field by field, and allows rules to be edited while the firewall runs, with a preview of the effect of each change.

The proposed solution enforces a clear security policy on a host, stops common attacks early, and makes the behaviour of a firewall visible and understandable to network administrators and students.

2. Problem Statement

Without a firewall, every open service on a host can be reached from any address on the network, exposing it to port scanning, brute-force logins, spoofed packets and denial-of-service floods. Simple stateless filters cannot tell a genuine reply from a forged packet, and the decisions of existing firewalls are hard to observe and explain. There is a need for a firewall that filters real traffic against a configurable policy, tracks connection state, detects and blocks common attacks automatically, and shows clearly why each packet was allowed or blocked.

3. Objectives

Capture real packets from the Linux kernel and parse their headers.

Filter packets with an ordered rule table and a default-deny policy.

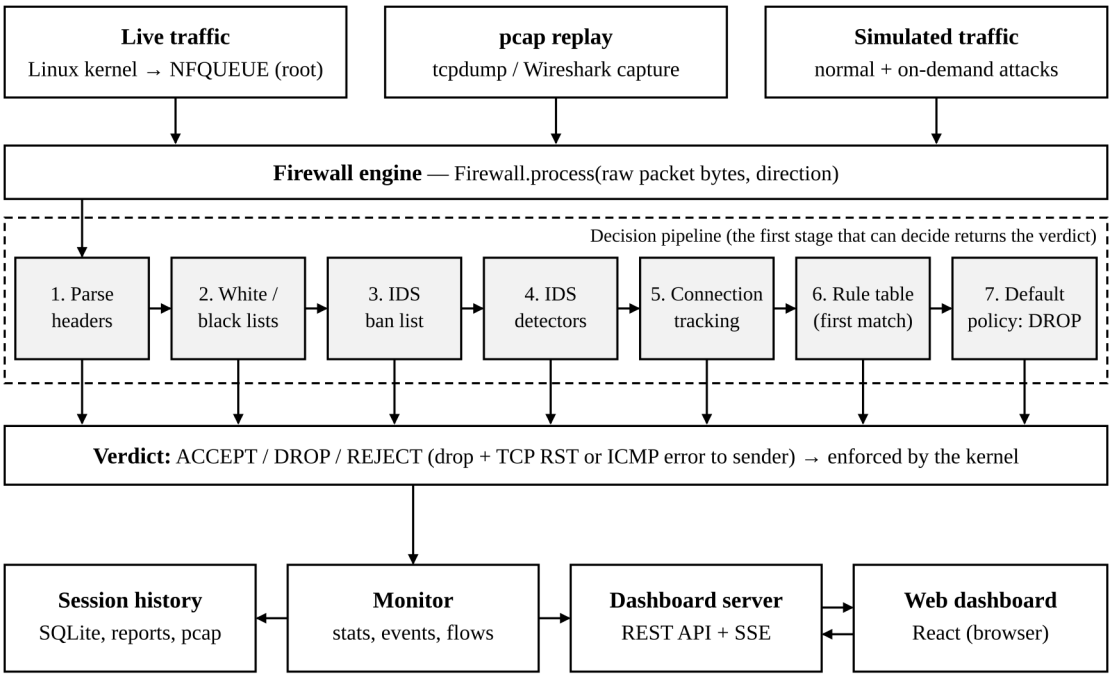
Track connections so that replies pass and spoofed packets are dropped.

Detect SYN floods, port scans and ping floods and ban the attacker.

Rate-limit selected traffic and log every verdict and security event.

Visualize traffic and explain every verdict through a web dashboard.

4. Proposed System Architecture



The system can be divided into the following modules:

Module	Description
Packet Capture	Receives packets from the kernel through NFQUEUE.
Packet Parsing	Decodes IPv4, IPv6, TCP, UDP and ICMP headers.
Rule Engine	Applies the ordered rule table; first match decides.
Connection Tracking	Classifies packets as NEW, ESTABLISHED or INVALID.
Intrusion Detection	Detects floods and port scans and bans the source.
Rule Analysis	Flags shadowed rules and previews rule changes.
Visualization	Displays live traffic, rules, analytics and alerts.
Report Generation	Saves history and exports session reports.

5. Technologies and Tools Required

Category	Tools used
Programming	Python 3 (standard library), TypeScript
Packet Capture	Linux netfilter, iptables, ip6tables, NFQUEUE, NetfilterQueue
Packet Analysis	tcpdump, Wireshark (pcap files)
Data Storage	SQLite, JSON rule file
Visualization	React, Vite, d3
Testing	Python unittest, network namespaces, veth pairs
Algorithms	First-match rule matching, token bucket, sliding-window detection

6. System Implementation

The project was implemented in six phases:

Packet Capture: Inserted iptables rules that send inbound and outbound traffic to NFQUEUE, so the kernel holds each packet until PyWall returns a verdict.

Header Parsing: Decoded IPv4, IPv6, TCP, UDP, ICMP and ICMPv6 headers from raw bytes and dropped malformed packets.

Rule Engine: Matched packets on direction, protocol, addresses, ports, ICMP type, TCP flags and rate limits, with ACCEPT, DROP and REJECT actions and a default-deny policy.

Connection Tracking: Classified each packet by its 5-tuple as NEW, ESTABLISHED or INVALID, giving replies a fast path and dropping spoofed packets.

Intrusion Detection: Counted SYNs, ports and pings per source in sliding windows and banned sources that crossed the thresholds for 60 seconds.

Dashboard Development: Developed an interactive dashboard displaying the traffic flow, packet details, connections, rules, analytics and security events.

7. Experimental Results and Performance Evaluation

The following results were obtained:

All 80 packets of 13 test scenarios got the expected verdict.

All 65 unit tests passed.

A real port scan and SYN flood were detected and banned.

Spoofed TCP packets were dropped by connection tracking.

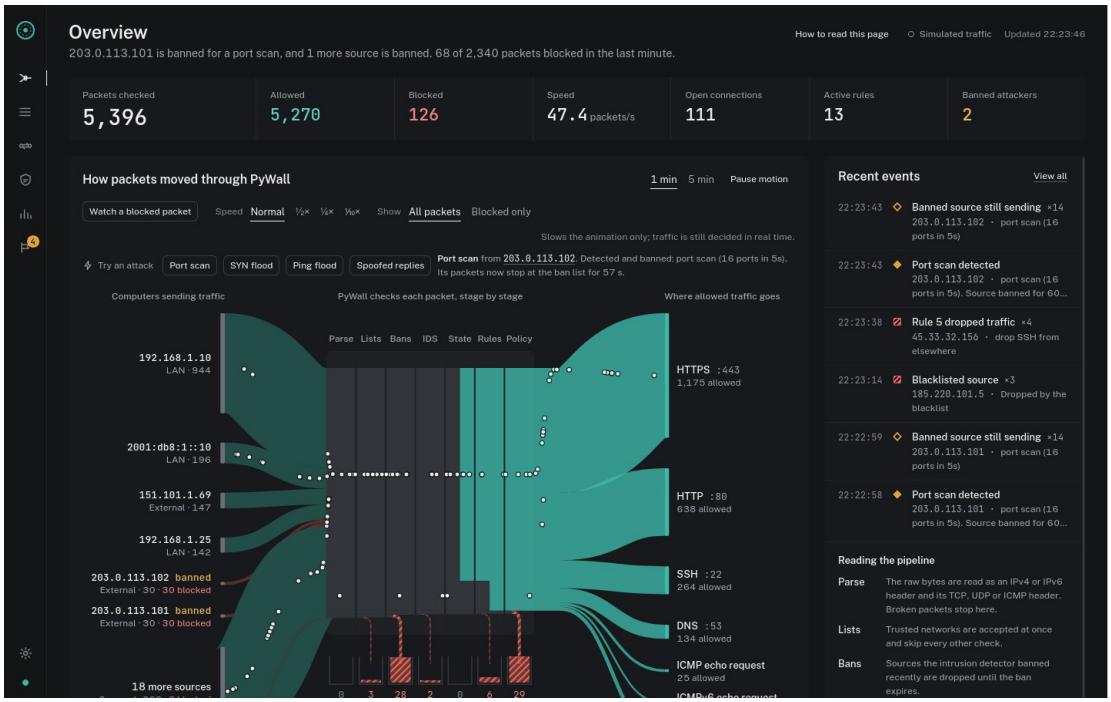
Excess ping traffic was dropped by the rate limit.

On a real network, Telnet was blocked while browsing worked.

Measured performance metrics:

Metric	Result
Accuracy	80 of 80 scenario packets correctly decided (100%)
Unit Tests	65 of 65 tests passed
Live Run	120 packets accepted, 122 dropped, 2 attackers banned
Processing Time	About 21 µs per packet in the firewall engine
Detection Time	Scan at 16 ports in 5 s; SYN flood at 21 SYNs in 1 s

8. Expected Output



The final system provides:

Real-time filtering of live network traffic.

Graphical flow map of traffic through the firewall.

Detection and banning of port scans and floods.

Explanation of the stage and rule behind each verdict.

Live rule editing with impact previews.

Downloadable session reports and blocked packets.

9. Conclusion

The Firewall Implementation project provides a working stateful packet-filtering firewall that filters real network traffic and enhances host security. By combining kernel packet capture, header parsing, an ordered rule table, connection tracking and intrusion detection, the system blocks unwanted connections, spoofed packets and common attacks while allowing legitimate traffic, and its dashboard makes every decision visible and explainable. The project provides practical exposure to network protocols, packet filtering, Linux networking, network security and data visualization.

10. References

The following references were used:

Netfilter Project – Netfilter documentation.

NetfilterQueue – Python NFQUEUE bindings.

RFC 791 – Internet Protocol.

RFC 8200 – IPv6 Specification.

RFC 9293 – Transmission Control Protocol.

Project Source Code – GitHub repository.

Live Demo – PyWall dashboard.

Chapter Contents

1	Introduction
2	Literature Review
3	Problem Definition and Objectives
4	Existing and Proposed Systems
5	System Architecture and Design
6	Methodology and Algorithms
7	Implementation
8	Results and Discussion
9	Performance Evaluation
10	Conclusion and Future Scope
	References and Appendices